

Part 01

- **Question:** What is the shortcut to comment and uncomment a selected block of code in Visual Studio?

- `ctrl + k + C` --> Comment
- `ctrl + k + u` --> Uncomment

- **Question:** Explain the difference between a runtime error and a logical error with examples.

- A **runtime error** happens **while the program is running**. The code may be syntactically correct, but something unexpected happens that prevents the program from continuing normally.

Example:

```
Int x = 10;  
Int y = 0;  
Int result = x/y;
```

Here the program tries to divide **x** by **0**, which causes a **DivideByZeroException** at runtime.

- A **logical error** happens when the program **runs successfully**, but gives the **wrong result** because the programmer's logic is incorrect.

Example:

```
Int x = 10;  
Int y = 20;  
Int sum = x - y  
Console.WriteLine(sum);
```

The program runs without crashing and prints:

-10

But if the purpose was to calculate the sum, the correct answer should be:

30

The mistake is that we used **-** instead of **+**.

- **Question:** Why is it important to follow naming conventions such as PascalCase in C#?

Naming conventions make code easier to read, understand, and maintain. They also make the code more consistent and professional, especially when working in a team.

- **Question:** Explain the difference between value types and reference types in terms of memory allocation.

- **Value types** store the actual value, and each variable has its own copy.
 - `Int` – `double` – `float` – `bool` – `char` – `struct`
- **Reference types** store a reference to an object in memory, so multiple variables can refer to the same object. Therefore, changing the object through one reference can affect the other references.
 - `string` – `class` – `array` – `object`

- **Question:** What will be the output of the following code? Explain why:

```
int a = 2, b = 7;  
Console.WriteLine(a % b);
```

The Output >> 2 //

- **Question:** How does the && (logical AND) operator differ from the & (bitwise AND) operator?

- **&& (Logical AND):**
 - Used with Boolean conditions (true or false).
 - Returns true only if both conditions are true.
 - Commonly used in if statements and logical expressions.
- **& (Bitwise AND):**
 - Used to perform an AND operation on the individual bits of numbers.
 - Compares each corresponding bit of two numbers.
 - Commonly used for bitwise operations.

Example:

```
bool result = true && false;    // false
```

```
int value = 5 & 3;              // 1
```

- **Question:** Why is explicit casting required when converting a double to an int?

- Explicit casting is required because converting a double to an int may cause data loss. The decimal part of the number will be removed during the conversion.
- Therefore, C# requires explicit casting to make the programmer aware that some data may be lost.

- **Question:** What exception might occur if the input is invalid and how can you handle it

- If the user enters invalid input, a **FormatException** may occur when using `int.Parse()`. We can handle this exception using a **try-catch block** to prevent the program from crashing and display an appropriate error message.

- **Question:** Given the code below, what is the value of x after execution? Explain why

```
int x = 5;  
int y = ++x + x++;
```

```
++x >> use x = 6 // First plus one then use it  
x++ >> use x = 6 // then add 1
```

```
y = 6 + 6 = 12  
x = 7
```